Neural Networks 201 (2026) 108926

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

FUGEA: Fused unified gradient ensemble for cross-architecture transferable attacks

Guangliang Huang, Feng Ye *, Tianqiang Huang, Chenhao Lu, Runze Chen, Quan Liu

Fujian Normal University, No. 18, Middle Wulongjiang Avenue, Fuzhou, 350117, Fujian Province, China

ARTICLE INFO

Keywords:
Adversarial attack
Image classification
Cross-architecture
Transfer attacks
Ensemble learning
Black-box attacks.

ABSTRACT

The transferability of adversarial examples is crucial for achieving black-box attacks in real-world scenarios. However, existing methods often exhibit limited effectiveness in terms of attack success rates and transferability across different model architectures (e.g., from CNN-based architectures to Vision Transformer (ViT)-based architectures). Such limitations impede reliable adversarial evaluation in safety-critical applications, such as autonomous driving and medical imaging. To address these challenges, we propose FUGEA, which decouples the adversarial attack process into a two-stage optimization designed to enhance cross-architecture transferability and attack success rates. In the first stage, the Rapid Convergence Engine (RCE) integrates Uncertainty Weight (UW) and Gradient Agreement Mapping (GAM) to dynamically allocate ensemble gradient weights and stabilize the optimization direction. In the second stage, the Precision Refinement Gradient (PRG) module employs Sampled Neighbor Predictive Gradient (SNPG) to achieve fine-grained perturbation optimization through the fusion of neighborhood sampling and future-step prediction. Extensive experiments demonstrate that FUGEA consistently outperforms state-of-the-art methods in attack success rates and transferability across both CNN and ViT architectures. Furthermore, it maintains robust performance against advanced adversarial defenses, providing a powerful and effective solution for real-world cross-architecture black-box attacks.

1. Introduction

Over the past decade, rapid advances in deep learning have reshaped computer vision, natural language processing, and related fields. Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) have driven substantial performance gains in image classification, object detection, and language generation. However, these models remain vulnerable to adversarial examples, namely carefully crafted inputs that cause incorrect predictions. This vulnerability reveals fundamental robustness limitations in deep neural networks Szegedy et al. (2014) and raises serious concerns about their reliability in safety critical applications such as image recognition and autonomous driving.

Despite notable progress in adversarial attack research, insufficient cross-architecture transferability remains a major challenge. Adversarial examples generated on one architecture often exhibit limited effectiveness when transferred to different architectures. For instance, examples crafted on CNN based models typically experience a significant drop in attack success rate when applied to Transformer based models. This performance degradation primarily stems from architectural differences: CNNs build global representations by hierarchically aggregating local features through convolutional kernels, whereas Transformers model long range dependencies via global attention from the outset, but lack the explicit local inductive bias of CNNs.

To improve the cross-architecture transferability of adversarial examples, various strategies have been proposed, including gradient based methods, input transformation techniques, and ensemble based attacks. Gradient based approaches, such as FGSM Goodfellow et al. (2015) and its iterative variants (I-FGSM Kurakin et al. (2017), MI-FGSM Dong et al. (2018), NI-FGSM Lin et al. (2020), PI-FGSM Gao et al. (2020), AI-FGTM Zou et al. (2022)), generate perturbations by exploiting model gradients. While these methods are generally effective across similar architectures, they tend to perform poorly when transferring adversarial examples between CNNs and ViTs. Input transformation methods (e.g., TIM Dong et al. (2019), SIM Lin et al. (2020), Admix Wang et al. (2021), PAM Zhang et al. (2023)) enhance perturbation diversity through data augmentation. However, the adversarial examples produced by these transformations often distort semantic information or critical features of the original input, resulting in adversarial examples that lack stealthiness.

In contrast, ensemble attacks enhance the transferability of adversarial examples by integrating gradients from multiple models, thereby

* Corresponding author.
E-mail addresses: 1970777408@qq.com (G. Huang), yefeng@fjnu.edu.cn (F. Ye), fjhtq@fjnu.edu.cn (T. Huang), 1838308983@qq.com (C. Lu), 1195997023@qq.com (R. Chen), 1438681822@qq.com (Q. Liu).

https://doi.org/10.1016/j.neunet.2026.108926
Received 9 September 2025; Received in revised form 30 March 2026; Accepted 31 March 2026
Available online 2 April 2026
0893-6080/© 2026 Published by Elsevier Ltd.

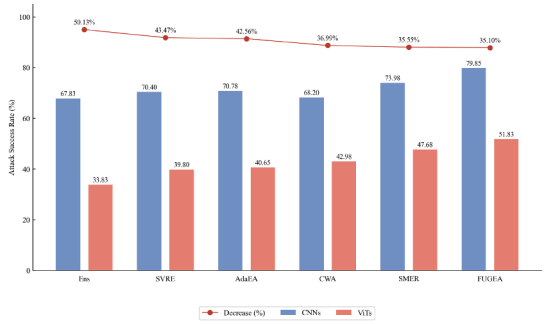


Fig. 1. A comparison of the adversarial attack transferability of adversarial examples generated by FUGEA and baseline methods through ensemble training on ResNet18, Inception_v3, BiT-M-R50×1, and Inception_v4. Our FUGEA achieves the best performance by optimizing ensemble gradients with a two-stage strategy.

capturing the vulnerabilities of various network architectures while preserving the semantic information of the input examples. However, existing ensemble based methods (e.g., Ens Liu et al. (2017), SVRE Xiong et al. (2022), AdaEA Chen et al. (2023), CWA Chen et al. (2024), and SMER Tang et al. (2024)) still face numerous limitations. For instance, Ens employs a simple uniform weighting scheme to calculate ensemble gradients, ignoring the differences between models and leading to poor performance in cross-architecture transfer attacks. SVRE overlooks the inherent heterogeneity among different surrogate models, resulting in a significant drop in attack success rates during cross-architecture transfers. Although AdaEA accounts for model discrepancies and attempts to stabilize the optimization direction, it still lacks fine-grained optimization; consequently, while it achieves decent cross-architecture transferability, its adversarial attack success rate remains insufficient. CWA relies on a static weight fusion strategy and fails to achieve dynamic adaptive fusion tailored to specific model features, leading to subpar performance in cross-architecture transferability. Finally, while SMER leverages reinforcement learning for weight allocation, it neglects the stability and fine-grained nature of the optimization trajectory, which leaves room for improvement in its attack success rate. Restricted by these factors, existing methods exhibit limited performance in terms of attack success rates and cross-architecture transferability, while also lacking sufficient generalization capabilities against diverse defense mechanisms.

To address these challenges, this paper proposes FUGEA, a novel adversarial attack framework designed to overcome the limitations of existing ensemble methods and enhance the cross-architecture transferability of adversarial examples. Aiming at the deficiencies of current methods in cross-architecture transferability and attack success rates, FUGEA introduces an innovative two-stage optimization paradigm consisting of a Rapid Convergence Engine (RCE) and a Precision Refinement Gradient (PRG) stage, allowing different stages to focus on optimizing distinct objectives.Specifically, in the RCE stage, we propose Uncertainty Weight (UW) and Gradient Agreement Mapping (GAM). By combining loss-based weights with gradient cross-influence weights, UW addresses the model heterogeneity issue that is often neglected by fixed-weight fusion methods. Simultaneously, GAM stabilizes the optimization of ensemble gradients by filtering directional inconsistencies; when coupled with a larger step size, it achieves a balance between rapidly approaching the decision boundary and maintaining directional stability.To further address the lack of fine-grained optimization in existing research, we introduce Sampled Neighbor Predictive Gradient (SNPG) in the PRG stage. This method predicts the next update by sampling perturbations around the current adversarial example, subsequently fusing the gradients of the sampled and predicted points with the loss gradient to achieve fine-grained ensemble optimization. These two stages work synergistically to generate adversarial examples with high attack success rates and robust transferability, enabling FUGEA to achieve superior performance across both CNNs and ViTs, as illustrated in Fig. 1.

The main contributions of this paper are summarized as follows:

- Proposal of the FUGEA Framework: To address the deficiencies of existing methods in weight allocation, stabilizing ensemble gradient optimization direction, and fine-grained optimization, we innovatively propose the FUGEA two-stage optimization framework consisting of RCE and PRG. This design effectively integrates the capabilities of the RCE stage in dynamic weight allocation and stable approximation of decision boundaries with the fine-grained optimization capabilities of the PRG stage, simultaneously enhancing both the success rate of adversarial attacks and cross-architecture transferability.
- Component Synergy Mechanism: FUGEA incorporates the UW, GAM, and SNPG strategies. Specifically, UW is proposed to address the shortcomings of existing methods in allocating weights across different surrogate models, ingeniously fusing loss weights and cross-weights to dynamically distribute weights based on the contribution of each model. GAM is introduced to solve the instability issues in ensemble gradient optimization direction faced by existing methods, stabilizing the optimization direction through the computation of a filtering matrix. SNPG is designed to overcome the limitations of existing methods in fine-grained optimization, achieving fine-grained optimization of ensemble gradients through sampling and predicting neighboring gradients. These components work synergistically to significantly improve attack success rates and cross-architecture transferability.
- Cross-Architecture Transferability Validation: Extensive experiments on CNNs and ViTs demonstrate that FUGEA consistently achieves excellent attack success rates and strong cross-architecture transferability. Furthermore, it demonstrates robust attack performance against advanced defense methods, validating its effectiveness across diverse settings and proving its practical viability in real-world attack scenarios.

2. Related work

2.1. Adversarial attacks

Adversarial attacks expose the vulnerability of deep learning models by introducing small, deliberately crafted perturbations to input data, causing the model to produce incorrect predictions Szegedy et al. (2014).

2.1.1. Gradient-based adversarial attacks.

Gradient-based adversarial attacks utilize gradient information from the model to craft adversarial perturbations by optimizing a loss function. The Fast Gradient Sign Method (FGSM) Goodfellow et al. (2015) is one of the earliest single-step attack algorithms, which efficiently generates perturbations using the sign of the gradient and highlights the linearity-induced vulnerabilities of deep models. FGSM was later extended to the Iterative FGSM (I-FGSM) Kurakin et al. (2017), which improves white-box attack success rates through iterative optimization. Momentum Iterative FGSM (MI-FGSM) Dong et al. (2018) introduces a momentum term to stabilize the update direction, thereby improving transferability to other models. Projected Gradient Descent (PGD) Madry et al. (2018), widely regarded as a strong baseline, iteratively generates adversarial examples via projected gradient descent and is commonly used in adversarial training. In recent years, researchers have proposed further refinements to enhance both the efficiency and effectiveness of gradient-based attacks. For instance, prediction-correction-based methods leverage numerical techniques from ordinary differential equations (ODEs) to improve attack trajectories Wan and Huang (2024), while Bayesian simulation-based ensemble attacks Li et al. (2023) significantly boost both success rate and transferability by modeling uncertainty across multiple surrogate models.

2.1.2. Input transformation-based adversarial attacks.

Adversarial attacks based on input transformations aim to improve the transferability of adversarial examples by applying random input manipulations or feature-level modifications. DI^2-FGSM enhances input diversity by performing random resizing and padding, thereby increasing cross-architecture transferability Xie et al. (2019). The Translation-Invariant Method (TIM) Dong et al. (2019) improves transferability by smoothing gradients using translation-invariant convolutional kernels. Admix Wang et al. (2021) improves transferability by mixing image features from different classes to create more misleading adversarial examples. Patch-level attacks, such as PI-FGSM Gao et al. (2020), emphasize critical local regions by applying perturbations selectively, increasing attack efficiency. Structure-Invariant Attack (SIA) Wang et al. (2023) applies random transformations to image blocks to generate structurally diverse inputs for gradient computation, significantly boosting transferability. Block-wise Spatial Rotation (BSR) Wang et al. (2024) perturbs and rotates local patches to disrupt attention mechanisms, thereby reducing heatmap consistency and enhancing the transferability of adversarial examples against different architectures.

2.1.3. Model ensemble attacks.

Ensemble-based adversarial attacks generate the adversarial examples by aggregating gradients or losses from multiple models, aiming to enhance generalization and cross-model transferability. The ensemble attack (Ens) framework Liu et al. (2017) averages the loss across several models to guide perturbation updates. To address gradient inconsistency, SVRE Xiong et al. (2022) introduces random variance into gradients, reducing gradient conflicts among models. Adaptive methods like AdaEA Chen et al. (2023) use variance filtering to dynamically reweight model contributions, improving cross-architecture transferability. SMER Tang et al. (2024) employs reinforcement learning to learn optimal weighting strategies that maximize attack loss. Meta-Gradient Attack (MGA) Yuan et al. (2021) applies meta-learning to optimize gradient fusion, significantly improving black-box attack performance. CWA Chen et al. (2024) considers both the flatness of the loss landscape and proximity to local optima to guide ensemble construction. Despite the advantages of ensemble-based methods, their performance remains limited in complex cross-architecture black-box settings, often exhibiting low attack success rates and weak transferability. These limitations highlight the need for more effective strategies that can jointly achieve high attack success and strong transferability across diverse architectures.

2.2. Adversarial defenses

To counter adversarial attacks, numerous defense strategies have been proposed to enhance the robustness of deep learning models against adversarial perturbations.

2.2.1. Adversarial training.

Adversarial training improves model robustness by incorporating adversarial examples into the training process. The standard approach involves generating adversarial examples online during training to expose the model to potential threats Madry et al. (2018). Ensemble adversarial training mitigates overfitting by leveraging adversarial examples generated from multiple models and incorporating them into the training set Tramèr et al. (2018). Curriculum Adversarial Training (CAT) gradually increases the strength of adversarial examples during training, guiding the model to learn robustness in a progressive manner Cai et al. (2018). Discrete adversarial training introduces discrete representation constraints to enhance the robustness of Vision Transformers (ViTs) against adversarial perturbations Mao et al. (2022). Fast adversarial training (FAT) combines random initialization with FGSM-based perturbations to accelerate training while maintaining acceptable robustness Wong et al. (2020).

2.2.2. Input preprocessing.

Input preprocessing techniques aim to mitigate adversarial effects by transforming or sanitizing input data before feeding it into the model. The Randomization and Padding (R&P) method disrupts the structure of adversarial noise via stochastic resizing and padding Xie et al. (2018). ComDefend employs a compression-reconstruction network to suppress adversarial perturbations Jia et al. (2019). Feature distillation-based methods leverage JPEG compression to remove noise while preserving semantic information Liu et al. (2019). High-level Guided Denoising (HGD) trains a denoiser using the target model's high-level feature representations to defend against both white-box and black-box attacks Liao et al. (2018). Randomized Smoothing (RS) provides certifiable robustness by averaging predictions over noisy inputs Cohen et al. (2019). Neural Representation Purification (NRP) enhances adversarial robustness by restoring clean representations through purification networks trained on adversarial examples Naseer et al. (2020). DiffPure (DP) proposes a novel adversarial purification method that leverages a pretrained diffusion model to remove adversarial perturbations and defend against adversarial attacks Nie et al. (2022).

3. Methodology

In the FUGEA method, the goal of the adversarial attack is to generate an adversarial example $x_{adv} = x + \delta$ by adding a well-designed perturbation δ to the original input x. This process aims to make the target model f produce false predictions while ensuring that the perturbations δ satisfy certain constraint conditions, e.g. $\| \delta \|_\infty \leq \epsilon$. An overview of FUGEA is shown in Fig. 2. To achieve efficient and highly transferable adversarial example generation, FUGEA employs a two-stage strategy: the RCE and the PRG, which iteratively optimizes the perturbation δ to maximize the loss function of the target model, where the perturbation δ is updated with the formula:

$$\delta_{t+1} = \mathrm{clip}\big(\delta_t + \alpha \cdot \mathrm{sign}\big(m_t\big), -\epsilon, \epsilon\big), \tag{1}$$

where α represents the step size, which is used to control the magnitude of each iteration. m_t represents the momentum term, which is used to fuse the current gradient and historical gradients. ϵ represents the perturbation constraint, ensuring that $\| \delta \|_\infty \leq \epsilon$, and $\mathrm{sign}(\cdot)$ is the sign function, ensuring that the update direction is consistent with the gradient. $\mathrm{clip}(\cdot)$ is the clipping function that ensures that δ is within $[-\epsilon, \epsilon]$. The final adversarial example obtained is $x_{adv} = \mathrm{clip}(x + \delta, 0, 1)$.

3.1. Rapid convergence engine

In adversarial optimization, the initial perturbation δ is typically far from the model's decision boundary, especially in high-dimensional image spaces where the loss landscape is often highly complex and non-convex. Traditional attack methods such as FGSM or PGD frequently face two major limitations in the early iterations: small step sizes leading to slow convergence, and instability in update directions caused by gradient noise. In ensemble attacks, gradients from different models may lead to unstable optimization directions due to inconsistency. These factors significantly reduce the efficiency of adversarial attacks. To overcome these challenges, we propose the Rapid Convergence Engine (RCE). RCE employs a step size amplification strategy to help the perturbation more effectively escape the flat regions of the loss surface. Its design is inspired by acceleration techniques in gradient descent optimization. However, overly large step sizes can lead to optimization instability. Therefore, to stabilize the optimization direction, RCE integrates two components: Uncertainty Weight (UW) and Gradient Agreement Mapping (GAM). The UW strategy dynamically adjusts the contribution weight of each model by fusing the loss values of individual models and cross-model gradients, thereby generating more robust ensemble weights. GAM calculates the cosine similarity of gradients between models, filtering out noisy directions and retaining only the common attack directions. These two mechanisms together improve

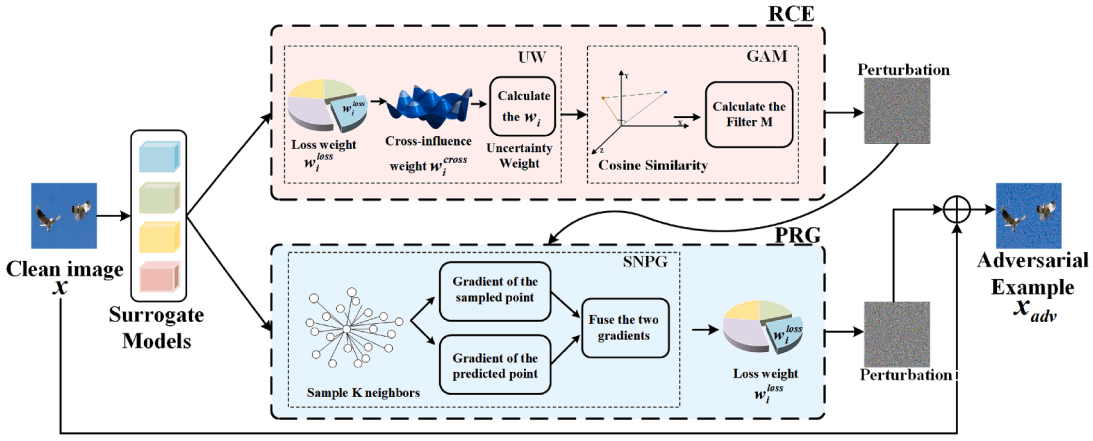


Fig. 2. An overview of FUGEA. The gradient obtained from the surrogate model is optimized through the RCE and PRG stages to generate adversarial examples with strong cross-architecture transferability.

the gradient quality, enhance the attack's targeting, and counteract the instability introduced by the step size amplification strategy. The introduction of RCE is aimed at acting as an accelerator, laying a solid foundation for subsequent detailed optimization.

The update formula for the perturbation in RCE is:

$$\delta_{t+1} = \mathrm{clip}\big(\delta_t + \alpha \cdot s \cdot \mathrm{sign}\big(m_t\big), -\epsilon, \epsilon\big), \tag{2}$$

where s is the step amplification factor, which accelerates the initial convergence, and $s > 1$. $m_t = \mu \cdot m_{t-1} + \nabla_\delta \mathcal{L}_{filtered}$ is the momentum term, where μ is the momentum decay factor, and $\nabla_\delta \mathcal{L}_{filtered}$ is the filtered ensemble gradient, computed via UW and GAM; $\mathcal{L} = CE(\sum_{i=1}^{N} w_i \cdot o_i, y)$ is the weighted cross-entropy loss, $o_i = f_i(x+\delta)$ is the output of the i-th model, and y is the true label, and w_i is the UW computed based on the cross-influence of loss and gradient, used to dynamically adjust the contribution of each model.

3.1.1. Uncertainty weight

Specifically, UW dynamically adjusts each model's contribution to enhance the success rate and transferability of adversarial examples. This mechanism combines model-specific loss values and gradient interactions to generate robust adaptive weights. The weights are assigned based on each model's loss values, with greater influence given to models with higher loss, as these models are considered more vulnerable and therefore more relevant to the current optimization objective.

First, we need to compute the weights based on the loss:

$$w_i^{\text{loss}} = \mathrm{softmax}(\mathcal{L}_i) = \frac{\exp(\mathcal{L}_i)}{\sum_{j=1}^{N} \exp(\mathcal{L}_j)}, \tag{3}$$

where $\mathcal{L}_i$ denotes the loss of the i-th model and N is the total number of models. Models with higher loss values are assigned greater weights to maintain and boost the attack success rate.

Then, the gradient cross-influence weight is calculated, considering the impact of the gradient direction from the i-th model on the loss of the j-th model, which measures the model's transferability. Models with better transferability, whose gradients result in larger losses in other models, are assigned higher weights:

$$w_i^{cross} = \mathrm{softmax}\Big(\sum_{j \neq i} \frac{\mathcal{L}_j(x_t + \alpha \cdot \mathrm{sign}(\nabla_\delta \mathcal{L}_i))}{\mathcal{L}_j(x_t + \alpha \cdot \mathrm{sign}(\nabla_\delta \mathcal{L}_j))} \cdot \beta\Big), \tag{4}$$

where x_t denotes the adversarial example in the current iteration, $\mathcal{L}_j(x_t + \alpha \cdot \mathrm{sign}(\nabla_\delta \mathcal{L}_i))$ represents the loss after updating the perturbation on j-th model using the gradient direction provided by the i-th model; $\mathcal{L}_j(x_t + \alpha \cdot \mathrm{sign}(\nabla_\delta \mathcal{L}_j))$ is the loss on j-th model after updating the perturbation using the gradient direction provided by itself; α is the step size, $\nabla_\delta \mathcal{L}_j$ is the gradient of the j-th model; and β is the scaling factor, used to adjust the weight of the gradient cross-influence.

Finally, the loss weights and the gradient cross-influence weights are combined to obtain the ensemble weights for integrating the success rate and transferability of the adversarial attack:

$$w_i = \mathrm{softmax}(w_i^{loss} \cdot w_i^{cross}). \tag{5}$$

3.1.2. Gradient agreement mapping

To enhance the stability of gradient optimization, we introduce the GAM component, which generates a filtering matrix M by computing pixel-level cosine similarities of gradients across models, thereby filtering out noisy directions while preserving shared attack directions. Specifically, cosine similarity is employed to quantify gradient alignment among different surrogate models. For each model, the channels at each pixel are treated as a single vector, and the cosine similarity between its gradient and those of all other models is calculated. These values are subsequently averaged over the model dimension to produce a cross-model gradient similarity map. This effectively suppresses inconsistent gradient directions across models, retaining only those demonstrating strong consensus. Consequently, adversarial perturbations are guided toward directions more likely to transfer successfully across diverse architectures, thereby enhancing both attack success rate and transferability.

First, we compute the gradient similarity map:

$$S_{\cos}{}^{B\times 1\times H\times W} = \frac{1}{N}\sum_{i=1}^{N}\Big(\frac{1}{N-1}\sum_{j\neq i}\cos(\nabla_\delta \mathcal{L}_i, \nabla_\delta \mathcal{L}_j)\Big), \tag{6}$$

where $cos(\cdot)$ denotes the cosine similarity function, $\nabla_\delta \mathcal{L}_i$ and $\nabla_\delta \mathcal{L}_j$ are the gradients of the model. Then the obtained cross model gradient similarity is used to get the filter M to remove the difference part in the fused gradient:

$$M^{B\times 1\times H\times W} = 1_{S_{\cos}\geq\theta}, \tag{7}$$

where θ is the threshold for gradient filtering, used to determine pixel-by-pixel whether the gradient should be retained. By filtering out the differential components in the ensemble gradient, the direction of gradient optimization can be synchronized.

Finally, the gradient is filtered using the w_i obtained from UW and the filtering matrix M obtained from GAM:

$$\nabla_\delta \mathcal{L}_{filtered} = \nabla_\delta \mathcal{L}\Big(\sum_{i=1}^{N} w_i \cdot f_i(x_t),\ y\Big) \odot M, \tag{8}$$

where N is the number of models; and $\odot$ denotes Element-wise multiplication.

Table 1
Comparison of attack success rates (%) between FUGEA and baseline methods. All adversarial examples are generated on CNN architecture surrogate models. Bolded numbers signify better results.

Base	Models	Attack	CNNs					ViTs				
			Res-50	Den-121	Res-101	BiT-101	Avg.	ViT-B	PiT-B	Vis-S	Swin-B	Avg.
MI-FGSM	Res-18 Inc-v3 BiT-50	Ens	62.0	75.2	49.7	62.8	62.43	28.3	35.8	47.7	27.8	34.90
		SVRE	62.1	77.3	51.1	63.4	63.48	29.4	36.3	48.8	30.7	36.30
		AdaEA	64.5	77.6	52.0	61.7	63.95	30.5	37.2	48.0	29.1	36.20
		CWA	59.3	73.4	44.9	60.5	59.53	22.7	27.7	39.8	21.5	27.93
		SMER	70.7	81.5	63.2	69.7	71.28	36.4	41.0	54.2	37.6	42.30
		FUGEA	**77.4**	**85.9**	**68.3**	**76.6**	**77.05**	**39.2**	**43.5**	**57.8**	**39.2**	**44.93**
	Res-18 Inc-v3 BiT-50 Inc-v4	Ens	66.9	78.7	57.8	67.9	67.83	26.7	33.7	48.3	26.6	33.83
		SVRE	70.5	79.7	61.7	69.7	70.40	33.3	40.1	51.2	34.6	39.80
		AdaEA	70.4	83.1	60.2	69.4	70.78	33.9	40.8	54.1	33.8	40.65
		CWA	69.0	82.9	53.6	67.3	68.20	35.9	43.6	56.7	35.7	42.98
		SMER	73.2	80.8	69.2	72.7	73.98	39.6	49.0	61.6	40.5	47.68
		FUGEA	**79.6**	**86.7**	**74.0**	**79.1**	**79.85**	**44.1**	**51.6**	**66.2**	**45.4**	**51.83**
I-FGSM	Res-18 Inc-v3 BiT-50	Ens	38.2	55.8	28.0	39.3	40.33	16.4	17.8	27.0	15.6	19.20
		SVRE	18.0	27.3	13.5	17.3	19.03	7.7	8.8	11.4	5.4	8.33
		AdaEA	38.9	56.1	27.1	37.7	39.95	14.5	18.3	25.9	15.2	18.48
		CWA	22.6	31.7	14.4	36.9	26.40	11.0	10.7	15.5	8.0	11.30
		SMER	63.8	78.9	51.6	66.2	65.13	27.2	34.9	47.5	28.5	34.53
		FUGEA	**65.1**	**80.4**	**56.3**	**67.7**	**67.38**	**31.4**	**36.3**	**49.0**	**31.5**	**37.05**
	Res-18 Inc-v3 BiT-50 Inc-v4	Ens	44.8	60.2	35.5	44.5	46.25	19.2	23.2	31.9	19.6	23.48
		SVRE	27.0	34.0	22.0	26.4	27.35	10.3	11.8	12.8	8.4	10.83
		AdaEA	47.4	64.5	35.7	45.0	48.15	18.6	23.2	33.5	18.9	23.55
		CWA	33.1	42.6	24.8	36.1	34.15	14.8	17.9	24.0	14.1	17.70
		SMER	66.3	75.8	56.4	63.6	65.53	28.4	37.9	49.4	29.3	36.25
		FUGEA	**68.3**	**78.2**	**58.9**	**66.9**	**68.08**	**33.5**	**39.7**	**51.8**	**32.9**	**39.48**

3.2. Precision refinement gradient

After the RCE module rapidly pushes the perturbation towards the decision boundary, the next step is to fine-tune the perturbation direction to improve the transferability of the generated adversarial examples. Traditional methods at this stage typically rely on single-step gradient updates, which are susceptible to local noise and model heterogeneity. As a result, these methods often perform poorly when attacking unseen or structurally different models.

To overcome this limitation, we introduce the Sampled Neighbor Predictive Gradient (SNPG) component, which samples multiple neighboring points around the current perturbation. Furthermore, to further enhance transferability, the sampled gradient of the current step is combined with the predicted future step gradient information, enabling the attack to be optimized from a broader perspective. These two strategies jointly fine-tune the perturbation with higher precision. As a result, the attack performance is significantly improved across different model architectures. The perturbation iteration formula in the PRG stage is shown in Eq. (1). where $m_t = \mu \cdot m_{t-1} + \nabla_\delta \mathcal{L}_{fusion}$ is the momentum term, where μ is the momentum decay factor, and $\nabla_\delta \mathcal{L}_{fusion}$ is the fused gradient computed via SNPG and loss weights.

3.2.1. Sampled neighbor predictive gradient

To enhance the stability of the gradient and mitigate local random disturbances, SNPG performs K random samples within the neighborhood in each iteration. For the k-th sampling, where $k \in \{1, 2, \ldots, K\}$, a sampling point is constructed by adding random noise p_k, sampled from a uniform distribution, to the current adversarial example x_t within the ζ-neighborhood of x_t:

$$x_t^k = x_t + p_k, \quad p_k \sim U(-\zeta, \zeta), \tag{9}$$

subsequently, calculate the local gradient of the sampled point x_t^k on the i-th surrogate model:

$$g_{i,k} = \nabla_\delta \mathcal{L}_i(x_t^k, y), \tag{10}$$

then, using the directional information of the sampled points, an optimization step with a step size of α is simulated to predict the future position after the perturbation evolves:

$$x_{pred}^k = \left(x_t^k + \alpha \cdot \text{sign}(\nabla_\delta \mathcal{L}_i(x_t^k, y))\right), \tag{11}$$

then, calculate the local gradient of the predicted point x_{pred}^k on the i-th surrogate model:

$$g_{i,k}^{pred} = \nabla_\delta \mathcal{L}_i(x_{pred}^k, y), \tag{12}$$

next, the gradients of the sampled points and predicted points are fused, and all K samplings are fused:

$$\nabla_\delta \mathcal{L}_{\text{SNPG},i} = \frac{1}{K} \sum_{k=1}^{K} \left((1-\gamma) \cdot g_{i,k} + \gamma \cdot g_{i,k}^{pred}\right), \tag{13}$$

where γ is the fusion coefficient, used for weighted fusion to optimize gradient updates and improve the transferability of adversarial attacks.

Finally, the weights based on the loss values w_i^{loss} are obtained through Eq. (3) and fused with $\nabla_\delta \mathcal{L}_{\text{SNPG},i}$ to obtain the final fused gradient:

$$\nabla_\delta \mathcal{L}_{fusion} = \sum_{i=1}^{N} w_i^{loss} \cdot \nabla_\delta \mathcal{L}_{\text{SNPG},i}. \tag{14}$$

The overall FUGEA procedure is shown in Algorithm 1.

4. Experiments

4.1. Experimental setting

4.1.1. Datasets

In this study, we construct the dataset by randomly sampling one image from each class in the ImageNet validation set, which contains 50,000 images across 1000 categories. This results in a 1,000-image subset that preserves the original class diversity while significantly reducing computational overhead.

4.1.2. Network architecture

We used models of both CNN and ViT architectures, models of CNN architecture include: ResNet18 (Res-18) He et al. (2016), BiT-M-R50×1 (BiT-50) Kolesnikov et al. (2020), Inception_v3 (Inc-v3) Szegedy et al. (2016), and Inception_v4 (Inc-v4) Szegedy et al. (2016); models of ViT architecture include: ViT-Tiny (ViT-T) Wu et al. (2022), DeiT-Tiny

Table 2
Comparison of attack success rates (%) between FUGEA and baseline methods. All adversarial examples are generated on ViT architecture surrogate models. Bolded numbers signify better results.

Base	Models	Attack	CNNs					ViTs				
			Res-50	Den-121	Res-101	BiT-101	Avg.	ViT-B	PiT-B	Vis-S	Swin-B	Avg.
MI-FGSM	ViT-T DeiT-T Swin-T	Ens	54.7	62.2	42.2	53.2	53.08	68.0	54.8	61.9	62.8	61.88
		SVRE	53.1	63.3	41.8	50.3	52.13	67.0	54.6	61.3	63.1	61.50
		AdaEA	52.6	64.7	41.7	55.3	53.58	68.7	53.5	62.0	61.3	61.38
		CWA	50.2	60.1	34.3	45.6	47.55	59.5	41.7	52.5	49.7	50.85
		SMER	56.3	70.7	48.6	61.0	59.15	73.7	58.9	65.2	66.5	66.08
		FUGEA	**60.4**	**73.7**	**50.6**	**63.6**	**62.08**	**79.9**	**59.1**	**67.9**	**69.9**	**69.20**
	ViT-T Swin-T DeiT-T ViT-S	Ens	57.9	67.4	49.0	59.3	58.40	85.2	63.4	68.2	75.0	72.95
		SVRE	61.1	72.1	53.4	62.1	62.18	82.5	64.6	69.1	77.5	73.43
		AdaEA	59.7	69.0	50.7	60.8	60.05	87.6	63.2	70.5	76.4	74.43
		CWA	56.0	69.9	45.1	54.6	56.40	82.0	52.4	63.4	67.0	66.20
		SMER	63.4	78.8	55.4	68.0	66.40	90.1	68.3	73.9	81.8	78.53
		FUGEA	**68.5**	**80.7**	**59.8**	**73.5**	**70.63**	**93.0**	**70.2**	**78.0**	**81.9**	**80.78**
I-FGSM	ViT-T DeiT-T Swin-T	Ens	26.7	38.9	19.6	29.4	28.65	37.3	28.8	35.0	36.0	34.28
		SVRE	22.8	27.5	15.1	25.7	22.88	25.2	21.4	26.1	23.1	23.95
		AdaEA	24.8	35.3	17.5	27.8	26.35	33.2	26.2	32.1	33.5	31.25
		CWA	29.2	38.6	21.0	30.1	29.73	22.3	26.8	36.6	36.6	30.58
		SMER	40.9	56.3	30.6	42.9	42.68	58.5	40.3	49.3	50.5	49.65
		FUGEA	**49.9**	**63.9**	**39.4**	**51.8**	**51.25**	**68.2**	**44.4**	**55.9**	**55.1**	**55.90**
	ViT-T Swin-T DeiT-T ViT-S	Ens	33.2	43.3	25.9	36.0	34.60	59.5	37.1	44.1	48.6	47.33
		SVRE	25.5	33.2	20.4	28.3	26.85	36.0	27.1	32.0	29.1	31.05
		AdaEA	32.4	42.2	23.7	35.9	33.55	56.0	35.0	41.5	48.4	45.23
		CWA	43.0	51.8	37.4	49.8	45.50	73.4	45.6	49.3	57.6	56.48
		SMER	48.1	61.2	39.7	50.5	49.88	75.8	46.5	59.3	62.3	60.98
		FUGEA	**54.7**	**67.2**	**43.1**	**57.5**	**55.63**	**80.6**	**52.4**	**60.5**	**65.7**	**64.80**

Table 3
Comparison of attack success rates (%) between FUGEA and baseline methods. All adversarial examples are generated on CNN and ViT architecture surrogate models. Bolded numbers signify better results.

Base	Models	Attack	CNNs					ViTs				
			Res-50	Den-121	Res-101	BiT-101	Avg.	ViT-B	PiT-B	Vis-S	Swin-B	Avg.
MI-FGSM	Res-18 ViT-T Inc-v3	Ens	60.3	74.9	49.2	57.5	60.48	45.6	42.1	53.8	38.0	44.88
		SVRE	64.5	78.2	51.9	57.2	62.95	43.2	41.8	53.3	36.9	43.80
		AdaEA	63.5	79.2	51.1	59.0	63.20	49.6	44.1	55.7	41.2	47.65
		CWA	58.9	77.5	44.4	52.7	58.38	40.7	34.0	43.9	29.9	37.13
		SMER	70.4	81.8	60.4	68.0	70.15	48.2	44.4	58.7	42.9	48.55
		FUGEA	**74.5**	**84.5**	**66.1**	**71.8**	**74.23**	**49.9**	**48.0**	**61.9**	**44.2**	**51.00**
	Inc-v3 ViT-T Swin-T	Ens	51.3	59.7	43.3	48.9	50.80	47.6	43.8	53.7	48.5	48.40
		SVRE	54.3	67.9	48.4	51.9	55.63	48.8	42.9	56.7	49.7	49.53
		AdaEA	50.6	59.9	42.0	50.1	50.65	50.1	45.7	55.2	50.7	50.43
		CWA	55.2	64.2	43.9	51.6	53.73	48.7	42.5	52.6	49.3	48.28
		SMER	62.1	70.0	51.4	59.5	60.75	49.4	46.7	58.6	50.1	49.95
		FUGEA	**67.5**	**78.2**	**61.4**	**64.4**	**67.88**	**50.3**	**50.2**	**62.0**	**51.0**	**53.38**
I-FGSM	Res-18 ViT-T Inc-v3	Ens	37.2	57.2	28.2	35.6	39.55	24.2	23.4	30.7	21.7	25.00
		SVRE	27.8	39.7	21.1	29.3	29.48	12.0	11.4	14.2	8.3	11.48
		AdaEA	39.8	57.1	28.7	37.1	40.68	26.1	23.1	31.0	20.2	25.10
		CWA	29.7	41.0	20.8	33.2	31.18	14.6	14.5	19.3	11.1	14.88
		SMER	58.2	69.9	49.0	61.3	59.60	32.5	35.1	43.5	31.4	35.63
		FUGEA	**63.7**	**79.4**	**52.2**	**62.0**	**64.33**	**38.3**	**38.5**	**48.9**	**34.4**	**40.03**
	Inc-v3 ViT-T Swin-T	Ens	29.9	37.1	22.0	29.1	29.53	23.8	23.1	29.0	25.3	25.30
		SVRE	26.1	32.6	21.3	28.8	27.20	15.7	13.0	18.5	13.2	15.10
		AdaEA	38.9	56.1	27.5	35.5	39.50	24.5	22.9	30.4	20.5	24.58
		CWA	35.9	45.0	26.8	36.4	36.03	18.2	21.4	26.8	17.1	20.88
		SMER	48.7	57.5	39.1	46.7	48.00	30.0	36.7	41.6	38.1	36.60
		FUGEA	**59.0**	**71.7**	**50.0**	**55.9**	**59.15**	**41.1**	**40.7**	**50.9**	**40.6**	**43.33**

(DeiT-T) Touvron et al. (2021), Swin-Tiny (Swin-T) Liu et al. (2021), and ViT-Small (ViT-S) Dosovitskiy et al. (2021). Similarly, for surrogate models, we chose models with CNN and ViT architectures, where models with CNN architecture include ResNet50 (Res-50) He et al. (2016), ResNet101 (Res-101) He et al. (2016), DenseNet121 (Den-121) Huang et al. (2017), and BiT-M-R101×1 (BiT-101) Kolesnikov et al. (2020); ViT architecture models include ViT-Base (ViT-B) Dosovitskiy et al. (2021), PiT-Base (PiT-B) Heo et al. (2021), Visformer-Small (Vis-S) Chen et al. (2021), and Swin-Base (Swin-B) Liu et al. (2021). To further validate the transferability of FUGEA, we employed the aforementioned advanced defense methods, including AT, HGD, NRP, RS, and DP, and evaluated the attack success rate of FUGEA against these defenses.

4.1.3. Baseline comparison

We compare the proposed FUGEA approach with pioneering attack models that are also based on ensemble, namely, Ens, SVRE, AdaEA,

Table 4
Comparison of attack success rates (%) of FUGEA and baseline methods against adversarial defense methods. All adversarial examples are generated on different combinations of surrogate models. Bolded numbers signify better results.

Models	Attack	Defense					
		FAT	HGD	NRP	RS	DP	Avg.
Res-18 Inc-v3 BiT-50	Ens	35.0	49.6	40.7	24.3	36.0	37.12
	SVRE	36.0	61.0	45.0	23.8	42.5	41.66
	AdaEA	35.2	49.4	41.0	24.8	37.8	37.64
	CWA	35.8	38.8	39.7	23.9	30.0	33.64
	SMER	36.3	60.2	46.2	24.5	44.7	42.38
	FUGEA	**37.2**	**69.6**	**52.7**	**27.5**	**52.4**	**47.88**
Res-18 Inc-v3 BiT-50 Inc-v4	Ens	36.8	53.1	43.8	25.3	37.1	39.22
	SVRE	36.3	69.0	48.8	24.4	49.0	45.50
	AdaEA	35.5	61.3	47.4	24.3	44.5	42.60
	CWA	36.1	52.1	42.8	24.6	34.0	37.92
	SMER	37.5	73.8	51.5	25.2	54.3	48.46
	FUGEA	**38.0**	**77.2**	**55.8**	**27.6**	**60.1**	**51.74**
ViT-T DeiT-T Swin-T	Ens	36.5	37.9	40.0	23.9	35.3	34.72
	SVRE	36.3	45.2	40.4	25.6	39.4	37.38
	AdaEA	37.0	39.4	41.4	24.6	37.3	35.94
	CWA	36.2	33.5	36.1	24.7	30.7	32.24
	SMER	37.3	47.3	41.6	25.8	41.5	38.70
	FUGEA	**37.7**	**48.7**	**42.8**	**27.3**	**43.2**	**39.94**
ViT-T Swin-T DeiT-T ViT-S	Ens	36.8	45.8	42.5	25.1	41.9	38.42
	SVRE	38.0	53.7	44.5	26.5	48.8	42.30
	AdaEA	36.8	47.2	43.0	25.7	42.3	39.00
	CWA	37.6	40.8	40.1	26.1	34.2	35.76
	SMER	38.1	55.1	45.9	27.1	49.8	43.20
	FUGEA	**38.7**	**57.3**	**48.5**	**28.7**	**51.7**	**44.98**
Res-18 ViT-T Inc-v3	Ens	36.8	45.8	42.5	25.1	41.5	38.34
	SVRE	35.8	55.5	42.3	24.7	41.0	39.86
	AdaEA	36.8	52.3	43.3	24.7	41.1	39.64
	CWA	36.4	43.8	38.5	25.3	32.9	35.38
	SMER	36.7	59.5	47.3	26.2	45.8	43.10
	FUGEA	**37.0**	**68.6**	**50.6**	**28.2**	**51.6**	**47.20**
Inc-v3 ViT-T Swin-T	Ens	34.7	41.9	36.8	22.0	35.6	34.20
	SVRE	35.3	50.0	38.5	23.2	38.2	37.04
	AdaEA	34.8	42.0	37.1	23.7	34.2	34.36
	CWA	35.5	41.6	38.6	24.3	34.3	34.86
	SMER	35.6	51.3	40.0	24.0	40.1	38.20
	FUGEA	**36.0**	**63.2**	**46.2**	**25.7**	**48.0**	**43.82**

Table 5
Ablation study of attack success rates (%) for the RCE and PRG stages in FUGEA. Bolded numbers signify better results.

Models	Attack	CNNs					ViTs				
		Res-50	Den-121	Res-101	BiT-101	Avg.	ViT-B	PiT-B	Vis-S	Swin-B	Avg.
Res-18,Inc-v3 BiT-50,Inc-v4	RCE	66.9	75.8	56.6	65.5	66.20	32.6	37.2	49.6	31.6	37.75
	PRG	71.9	83.0	64.9	72.0	72.95	43.1	47.8	59.2	41.7	47.95
	FUGEA	**79.6**	**86.7**	**74.0**	**79.1**	**79.85**	**44.1**	**51.6**	**66.2**	**45.4**	**51.83**

Table 6
Ablation study of attack success rates (%) for different components in FUGEA. Bolded numbers signify better results.

Models	Attack	CNNs					ViTs				
		Res-50	Den-121	Res-101	BiT-101	Avg.	ViT-B	PiT-B	Vis-S	Swin-B	Avg.
Res-18,Inc-v3 BiT-50,Inc-v4	**FUGEA**	**79.6**	**86.7**	**74.0**	**79.1**	**79.85**	**44.1**	**51.6**	**66.2**	**45.4**	**51.83**
	w/o UW	70.1	78.6	65.8	72.8	71.83	37.3	44.5	58.7	37.8	44.58
	w/o GAM	78.6	85.2	73.2	77.8	78.70	43.2	51.0	65.1	44.3	50.90
	w/o SNPG	61.3	74.6	51.7	60.0	61.90	28.5	34.4	46.5	28.9	34.58

CWA, and SMER. All ensemble methods follow the same ensemble setup in the experiments.

4.1.4. Implementation details

All experiments are conducted on NVIDIA GeForce RTX 3090 GPUs (24GB VRAM) to ensure consistency in computational performance. For the baseline model and FUGEA, we use MI-FGSM with 15 iterations under the constraint of l_∞ as the basic attack method and set $\epsilon = 16/255$, $\alpha = 1.6/255$, $\mu = 1.0$ in the generation of adversarial example, and for the hyperparameters we set $\beta = 10$ for the computation of the UW, $\theta = -0.3$ used by the GAM, $s = 10$ and $T_{RCE} = 5$ used by the RCE. In PRG, set $T_{PRG} = 10$ and SNPG uses $K = 20$, $\gamma = 0.5$, and $\zeta = 3\epsilon$. For AT, we use the version trained on Res-50 with $\epsilon = 4/255$ as provided in the original paper. For HGD and NRP, we use the methods provided in the

Table 7
Ablation study of attack success rates (%) for different iteration distribution in FUGEA.

Models	Iterations	CNNs					ViTs				
		Res-50	Den-121	Res-101	BiT-101	Average	ViT-B	PiT-B	Vis-S	Swin-B	Average
Res-18,Inc-v3 BiT-50,Inc-v4	2+8	71.1	79.1	65.0	71.0	71.55	39.4	45.9	56.2	39.8	45.33
	4+6	75.6	83.5	69.2	76.5	76.20	40.9	48.3	62.0	41.2	48.10
	5+10	79.6	86.7	74.0	79.1	79.85	44.1	51.6	66.2	45.4	51.83
	6+9	79.2	56.7	73.6	78.7	79.55	42.4	50.9	65.8	44.8	50.98
	10+10	83.8	89.6	76.3	79.9	82.40	44.9	53.4	67.7	47.0	53.25

Algorithm 1 FUGEA.

Require: clean image x, a list of N surrogate models $\{f_i\}_{i=1}^{N}$, perturbation bound ϵ, step size α, scaling factor β, threshold θ, step amplification factor s, momentum decay factor μ, fusion coefficient γ, sampling range ζ, RCE iterations T_{RCE}, PRG iterations T_{PRG}, number of samples K. LASTCON x^{adv}

Ensure:

Initialize $\delta \leftarrow 0$, $m \leftarrow 0$
for $t = 1$ **to** T_{RCE} **do**
  Compute the loss-based weights ${w_i}^{loss}$ using Eq. (3)
  Compute the gradient cross-influence weights ${w_i}^{cross}$ using Eq. (4)
  Compute the uncertainty weights w_i using Eq. (5)
  Compute the gradient similarity map S_{cos} using Eq. (6)
  Compute the filter matrix M using Eq. (7)
  Compute the filtered gradient $\nabla_\delta \mathcal{L}_{filtered}$ using Eq. (8)
  Update Perturbation δ_{t+1} using Eq. (2)
end for
PRGPHASE
for $t = 1$ **to** T_{PRG} **do**
  Compute the sampling points and their corresponding gradients using Eqs. (9)- (10)
  Compute the predicted points and their corresponding gradients using Eqs. (11)- (12)
  Compute the SNPG gradient $\nabla_\delta \mathcal{L}_{\text{SNPG},i}$ using Eq. (13)
  Compute the fusion gradient $\nabla_\delta \mathcal{L}_{fusion}$ using Eq. (14)
  Update Perturbation δ_{t+1} using Eq. (1)
end for
$x^{\text{adv}} \leftarrow \text{clip}_{[0,1]}(x + \delta)$

original paper and conduct experiments on Res-50. For RS, we choose the version with $\sigma = 0.5$ provided in the original paper and conduct experiments on Res-50. For DP, we choose the Score-SDE model with timestep = 150 and conduct experiments on Res-101. The other hyperparameters of Ens, SVRE, AdaEA, CWA, and SMER we refer to the settings of the original paper.

4.2. Main results

4.2.1. Ensemble adversarial attacks based on CNNs

We first evaluated the performance of FUGEA after training on models based on the CNN architecture and conducted experiments on both CNN and ViT architectures. The experimental results are shown in Table 1. It can be observed that SMER, with the help of reinforcement learning, achieved good average adversarial attack success rates across models from different architectures. However, in comparison, our FUGEA achieved better results, obtaining optimal performance on both CNN and ViT architecture models through a two-stage optimization process with different focuses. In particular, when attacking CNN architecture models, the average adversarial attack success rate reached as high as 79.85%, while it also maintained the highest average adversarial attack success rate when attacking ViT architecture models, demonstrating strong transferability of adversarial attacks.

4.2.2. Ensemble adversarial attacks based on vits

We then evaluated the performance of FUGEA after training on models based on the ViT architecture. Similarly, experiments were conducted on both CNN and ViT architecture models. The experimental results are shown in Table 2. It can be observed that FUGEA outperformed SMER, achieving optimal results. In particular, when attacking ViT architecture models, the average adversarial attack success rate reached as high as 80.78%. At the same time, when attacking CNN architecture models, the best result obtained was 70.63%. This demonstrates that the RCE and PRG we designed successfully achieved high adversarial attack success rates and transferability through stable gradient optimization directions.

4.2.3. Ensemble adversarial attacks based on heterogeneous models

We also evaluated the performance of FUGEA after training on models based on both CNN and ViT architectures. Similarly, experiments were conducted on both CNN and ViT architecture models. As shown in Table 3, FUGEA achieved an average adversarial attack success rate higher than all baseline methods. Especially based on the I-FGSM optimization, the average adversarial attack success rate for both CNN and ViT models exceeded the second-best method by more than 6%. When attacking CNN architecture models, the highest average adversarial attack success rate reached 74.23%. At the same time, when attacking ViT architecture models, the best result obtained was 51.00%.

4.2.4. Attack advanced defense models

We also evaluated the performance of the FUGEA method against advanced adversarial defense techniques, with the results shown in Table 4. FUGEA consistently outperformed baseline methods across various defense mechanisms, demonstrating a clear advantage. After training on models based on different architectures, it achieved the optimal average adversarial attack success rate. This proves that the adversarial examples generated by FUGEA not only demonstrate excellent attack success rates and transferability on standard surrogate models but also achieve optimal performance when facing adversarial defense methods. It is precisely through FUGEA's use of UW, GAM, and SNPG strategies that stable optimization directions are obtained, which enhance the transferability of the adversarial examples.

4.2.5. Visualization of attack performance

Finally, to intuitively demonstrate the effectiveness of adversarial attacks, we visualized the heatmaps of clean images and adversarial examples under both white-box and black-box models, using adversarial examples trained on Res-18, ViT-T, and Inc-v3, as shown in Fig. 3. Figs. 3(b) and 3(d) show the differences in attack effects when facing white-box models. Like other attack methods, FUGEA misleads the model's judgment, but compared to other methods, FUGEA causes the model to focus more on irrelevant areas. Figs. 3(c) and 3(e) illustrate the differences in attack effects under black-box settings. In this case, the advantage of FUGEA becomes more pronounced: compared with



Fig. 3. Heatmaps under various white-box and black-box models. (a) Input images, including clean images and adversarial examples generated by different methods. (b-e) Heatmaps corresponding to Res-18, Res-50, ViT-T, and ViT-B, respectively.

adversarial examples generated by other methods, the black-box models under FUGEA attacks concentrate more on irrelevant regions. These visualizations further validate the superiority of adversarial examples generated by FUGEA in both attack effectiveness and cross-model transferability.

4.3. Ablation studies

In this subsection, we conduct a series of ablation experiments to investigate the impact of different stages, strategies, and hyperparameters in FUGEA.

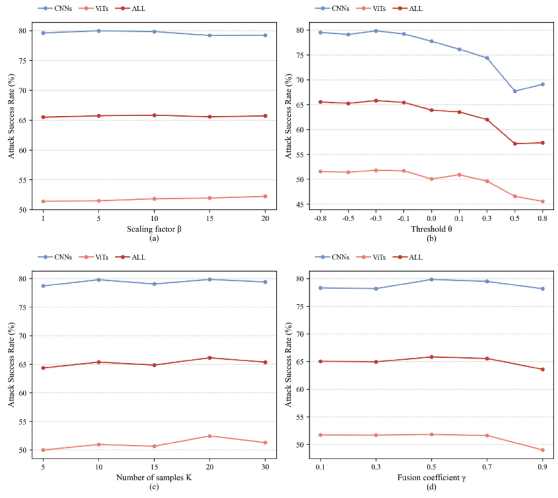


Fig. 4. Ablation study of the scaling factor β, threshold θ, number of samples K, and fusion coefficient γ.

4.3.1. Stages of FUGEA

To validate the individual contributions of each stage in the FUGEA framework, we conducted 15 rounds of iterative experiments under consistent hyperparameter settings. The ablation results are shown in Table 5. When only the RCE stage is used, the average attack success rates on CNN and ViT architectures drop to 66.2% and 37.75%, respectively, indicating that merely accelerating convergence to the decision boundary without subsequent fine-tuning severely limits the attack effectiveness. In contrast, when only the PRG stage is used, the success rates increase to 72.95% and 47.95%, showing some improvement but still falling short of the full model, and the training time is also increased. This confirms that the RCE stage quickly guides the perturbation towards the decision boundary, laying a necessary foundation for effective optimization in the PRG stage. When both stages are combined, FUGEA achieves success rates of 79.85% and 51.83% on CNN and ViT models, respectively, demonstrating the advantage of their complementary roles. These results prove that the collaborative two-stage design is crucial to both the overall attack performance and cross-architecture attack transferability.

4.3.2. Components of FUGEA

To further analyze the independent roles of different components in FUGEA, we conducted ablation experiments on UW, GAM, and SNPG. As shown in Table 6, removing UW leads to a significant decrease in adversarial attack success rates, which demonstrates the positive impact of our strategy of combining loss weights and gradient cross-influence weights to enhance attack performance. Although the performance drop is not as pronounced when GAM is removed, the final results still confirm that without GAM adjusting the optimization direction through the filtering matrix, relying solely on UW and SNPG cannot fully realize FUGEA's attack capability. Removing SNPG caused a significant drop in adversarial attack success rates, highlighting the importance of SNPG for both attack success rate and transferability. This is because SNPG, by merging the gradients of neighborhood sampling points and the predicted point gradient, provides a deeper understanding of the model's weaknesses, while also improving the success rate and transferability of the generated adversarial examples. Overall, the experimental results show that the collaboration among all components in FUGEA is indispensable.

4.3.3. Hyperparameters of FUGEA

We also studied the impact of hyperparameters on FUGEA, with the results shown in Fig. 4. Specifically, during the RCE stage, the scaling factor β, used by UW and the threshold θ used by GAM were investigated. In the PRG stage, the number of samples K used by SNPG and the fusion coefficient γ were examined. Among them, the value of β has little effect on the attack performance, but the optimal value was achieved when $\beta = 10$. The threshold θ is highly sensitive, and experiments show that an excessively large θ leads to a dramatic decline in attack effectiveness. This is because too large a θ filters out effective gradients. The number of samples K also caused visible fluctuations in the attack results; both very large and very small K values affected the adversarial attack performance. Different values of the fusion coefficient γ brought some fluctuations; specifically, values of $\gamma = 0.5$ and $\gamma = 0.7$ yielded good results, but values that were too large or too small negatively affected the attack. This suggests that both the current gradient of the sample points and the predicted gradients are important. Moreover, when $\gamma = 0.7$, it outperforms $\gamma = 0.3$, indicating that fusing the gradients of the predicted points greatly contributes to improving the success rate and cross-architecture transferability of adversarial attacks. Based on the experimental results, we finally chose the hyperparameters from the experimental setting.

4.3.4. Iterations of FUGEA

To further validate the rationality and fairness of the experimental setup, we conducted in-depth ablation studies on the iteration distribution within the two-stage optimization framework. As shown in Table 7, we analyzed the variation in attack performance under different total iteration counts and stage allocation ratios. When the total number of iterations was fixed at 10, the adversarial examples achieved an average attack success rate of merely 71.55% on CNN architectures and 45.33% on ViT architectures when $T_{\text{RCE}} = 2$ and $T_{\text{PRG}} = 8$. By adjusting the ratio to $T_{\text{RCE}} = 4$ and $T_{\text{PRG}} = 6$, the average attack success rates improved by 5.5% and 2.77%, respectively. Combined with the experimental data in Table 5, this demonstrates that the two-stage optimization of RCE and PRG constitutes a meticulously designed complementary mechanism; relying solely on either RCE or PRG fails to yield optimal results, and inappropriate allocation of optimization iterations likewise precludes peak performance. The experimental results reveal that when the total iteration count increased to 15, particularly when $T_{\text{RCE}} = 5$ and $T_{\text{PRG}} = 10$, the average attack success rates further improved by 3.65% and 3.73% compared to the $T_{\text{RCE}} = 4$ and $T_{\text{PRG}} = 6$ configuration. Notably, the performance improvement in cross-architecture transfer attacks on ViT models even surpassed that of same-architecture transfer attacks on CNN models. Furthermore, the setting of $T_{\text{RCE}} = 5$ and $T_{\text{PRG}} = 10$ outperformed that of $T_{\text{RCE}} = 6$ and $T_{\text{PRG}} = 9$, indicating that after RCE stabilizes the initial gradient optimization direction, PRG plays a crucial role in enhancing the transferability and attack success rate of adversarial examples. These experimental results corroborate the original design intention of FUGEA. Finally, although the setting of $T_{\text{RCE}} = 10$ and $T_{\text{PRG}} = 10$ could continue to improve the success rate, its marginal gain diminished compared to the improvement observed when increasing from 10 to 15 total iterations. Therefore, to balance computational efficiency and maximally demonstrate the novelty and transferability of FUGEA's two-stage optimization strategy, we ultimately set the default iteration counts to $T_{\text{RCE}} = 5$ and $T_{\text{PRG}} = 10$.

5. Conclusion

To address the limitations of existing methods, this paper proposes a novel adversarial attack framework, FUGEA, which significantly enhances the attack success rates and cross-architecture transferability of adversarial examples through a carefully designed two-stage optimization strategy. The core innovation of FUGEA lies in its integrated two-stage optimization scheme incorporating the UW, GAM, and SNPG strategies. Specifically, UW dynamically allocates weights based on the contribution of different surrogate models, GAM optimizes the optimization direction of ensemble gradients, and SNPG achieves fine-grained optimization of ensemble gradients by fusing sampled and predicted

neighborhood gradients. These strategies work synergistically to achieve a robust balance between improving attack success rates and enhancing cross-architecture transferability. Experimental results demonstrate that FUGEA exhibits excellent performance on both CNN and ViT architectures, outperforming existing methods in terms of attack success rates and cross-architecture transferability. Furthermore, FUGEA not only achieves high attack success rates on standard surrogate models but also effectively attacks advanced defense mechanisms. Its strong performance highlights the value of cross-architecture generalization capabilities in security-critical applications of adversarial attacks.

CRediT authorship contribution statement

Guangliang Huang: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing - original draft, Writing - review & editing; **Feng Ye:** Writing - review & editing; **Tianqiang Huang:** Writing - review & editing; **Chenhao Lu:** Supervision, Validation, Writing - review & editing; **Runze Chen:** Data curation; **Quan Liu:** Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was funded by the Collaborative Innovation Platform Project of Fuxiaquan National Independent Innovation Demonstration Zone (Project Number: 2023-P-003) and Key Special Project for Education Examination Enrollment in Fujian Province in 2024 (Project No. FJJKKS24-28).

References

Cai, Q.-Z., Liu, C., & Song, D. (2018). Curriculum adversarial training. In *International joint conference on artificial intelligence (IJCAI)*.

Chen, B., Yin, J.-L., Chen, S., Chen, B., & Liu, X. (2023). An adaptive model ensemble adversarial attack for boosting adversarial transferability. In *Ieee/cvf international conference on computer vision (iccv)*.

Chen, H., Zhang, Y., Dong, Y., Yang, X., Su, H., & Zhu, J. (2024). Rethinking model ensemble in transfer-based adversarial attacks. In *International conference on learning representations (ICLR)*.

Chen, Z., Xie, L., Niu, J., Liu, X., Wei, L., & Tian, Q. (2021). Visformer: The vision-friendly transformer. In *Ieee/cvf international conference on computer vision (iccv)*.

Cohen, J., Rosenfeld, E., & Kolter, J. Z. (2019). Certified adversarial robustness via randomized smoothing. In *International conference on machine learning (ICML)*.

Dong, Y., Liao, F., Pang, T., Su, H., Zhu, J., Hu, X., & Li, J. (2018). Boosting adversarial attacks with momentum. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Dong, Y., Pang, T., Su, H., & Zhu, J. (2019). Evading defenses to transferable adversarial examples by translation-invariant attacks. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International conference on learning representations (ICLR)*.

Gao, L., Zhang, Q., Song, J., Liu, X., & Shen, H. T. (2020). Patch-wise attack for fooling deep neural network. In *European conference on computer vision (ECCV)*.

Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International conference on learning representations (ICLR)*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Heo, B., Yun, S., Han, D., Chun, S., Choe, J., & Oh, S. J. (2021). Rethinking spatial dimensions of vision transformers. In *Ieee/cvf international conference on computer vision (iccv)*.

Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Jia, X., Wei, X., Cao, X., & Foroosh, H. (2019). Comdefend: An efficient image compression model to defend adversarial examples. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Kolesnikov, A., Beyer, L., Zhai, X., Puigcerver, J., Yung, J., Gelly, S., & Houlsby, N. (2020). Big transfer (bit): General visual representation learning. In *European conference on computer vision (ECCV)*.

Kurakin, A., Goodfellow, I. J., & Bengio, S. (2017). Adversarial examples in the physical world. In *International conference on learning representations (ICLR)*.

Li, Q., Guo, Y., Zuo, W., & Chen, H. (2023). Making substitute models more bayesian can enhance transferability of adversarial examples. In *International conference on learning representations (ICLR)*.

Liao, F., Liang, M., Dong, Y., Pang, T., Hu, X., & Zhu, J. (2018). Defense against adversarial attacks using high-level representation guided denoiser. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Lin, J., Song, C., He, K., Wang, L., & Hopcroft, J. E. (2020). Nesterov accelerated gradient and scale invariance for adversarial attacks. In *International conference on learning representations (ICLR)*.

Liu, Y., Chen, X., Liu, C., & Song, D. (2017). Delving into transferable adversarial examples and black-box attacks. In *International conference on learning representations (ICLR)*.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *Ieee/cvf international conference on computer vision (iccv)*.

Liu, Z., Liu, Q., Liu, T., Xu, N., Lin, X., Wang, Y., & Wen, W. (2019). Feature distillation: DNN-oriented JPEG compression against adversarial examples. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In *International conference on learning representations (ICLR)*.

Mao, X., Chen, Y., Duan, R., Zhu, Y., Qi, G., Ye, S., Li, X., Zhang, R., & Xue, H. (2022). Enhance the visual representation via discrete adversarial training. In *Advances in neural information processing systems (neurIPS)*.

Naseer, M., Khan, S. H., Hayat, M., Khan, F. S., & Porikli, F. (2020). A self-supervised approach for adversarial robustness. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Nie, W., Guo, B., Huang, Y., Xiao, C., Vahdat, A., & Anandkumar, A. (2022). Diffusion models for adversarial purification. In *International conference on machine learning (ICML)*.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I. J., & Fergus, R. (2014). Intriguing properties of neural networks. In *International conference on learning representations (ICLR)*.

Tang, B., Wang, Z., Bin, Y., Dou, Q., Yang, Y., & Shen, H. T. (2024). Ensemble diversity facilitates adversarial transferability. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., & Jégou, H. (2021). Training data-efficient image transformers & distillation through attention. In *International conference on machine learning (ICML)*.

Tramèr, F., Kurakin, A., Papernot, N., Goodfellow, I. J., Boneh, D., & McDaniel, P. D. (2018). Ensemble adversarial training: Attacks and defenses. In *International conference on learning representations (ICLR)*.

Wan, C., & Huang, F. (2024). Adversarial attack based on prediction-correction. *Expert Systems with Applications*, .

Wang, K., He, X., Wang, W., & Wang, X. (2024). Boosting adversarial transferability by block shuffle and rotation. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Wang, X., He, X., Wang, J., & He, K. (2021). Admix: Enhancing the transferability of adversarial attacks. In *Ieee/cvf international conference on computer vision (iccv)*.

Wang, X., Zhang, Z., & Zhang, J. (2023). Structure invariant transformation for better adversarial transferability. In *Ieee/cvf international conference on computer vision (iccv)*.

Wong, E., Rice, L., & Kolter, J. Z. (2020). Fast is better than free: Revisiting adversarial training. In *International conference on learning representations (ICLR)*.

Wu, K., Zhang, J., Peng, H., Liu, M., Xiao, B., Fu, J., & Yuan, L. (2022). Tinyvit: Fast pretraining distillation for small vision transformers. In *European conference on computer vision (ECCV)*.

Xie, C., Wang, J., Zhang, Z., Ren, Z., & Yuille, A. L. (2018). Mitigating adversarial effects through randomization. In *International conference on learning representations (ICLR)*.

Xie, C., Zhang, Z., Zhou, Y., Bai, S., Wang, J., Ren, Z., & Yuille, A. L. (2019). Improving transferability of adversarial examples with input diversity. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Xiong, Y., Lin, J., Zhang, M., Hopcroft, J. E., & He, K. (2022). Stochastic variance reduced ensemble adversarial attack for boosting the adversarial transferability. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Yuan, Z., Zhang, J., Jia, Y., Tan, C., Xue, T., & Shan, S. (2021). Meta gradient adversarial attack. In *Ieee/cvf international conference on computer vision (iccv)*.

Zhang, J., Huang, J.-t., Wang, W., Li, Y., Wu, W., Wang, X., Su, Y., & Lyu, M. R. (2023). Improving the transferability of adversarial samples by path-augmented method. In *Ieee/cvf conference on computer vision and pattern recognition (cvpr)*.

Zou, J., Duan, Y., Li, B., Zhang, W., Pan, Y., & Pan, Z. (2022). Making adversarial examples more transferable and indistinguishable. In *Aaai conference on artificial intelligence (aaai)*.